

RISING BEYOND BORDERS

Document 18 — Content & Editorial Architecture
Master Specification + Claude Code Implementation Prompt

Status: Frontend / repository-based content specification

Scope: This document defines how RBB content is structured, validated, approved and published inside the existing frontend architecture. **No CMS, admin dashboard, database-backed editor or admin route is included in this phase.**

1. Phase Objective

The RBB website currently follows a repository-backed content model: content source files feed React, the production build pre-renders public pages, and approved output is deployed as static HTML/assets. This phase formalizes that model so content can be added safely without inventing facts, publishing drafts accidentally or creating inconsistent page data.

2. Explicit Non-Scope

- No CMS.
- No admin dashboard.
- No /admin routes.
- No login/authentication system for editors.
- No content database.
- No browser-based content editor.
- No API for editorial content.
- No automatic publishing workflow.
- No change to the existing production hosting architecture.

3. Current Content Architecture

The implementation should continue using centralized frontend content modules under **src/content/**. Existing modules established by earlier documents remain the source of truth for their respective domains.

- homepage.js — homepage-specific structured content
- about.js — About content
- work.js — programs and project records
- impact.js — impact metrics, geography, approach and measurement
- getInvolved.js — involvement paths
- stories.js — story records
- contact.js — contact/careers content
- transparency.js — financial/transparency content
- team.js — team records
- donation.js — donation/giving content
- policies.js — policy content

- seo.js — shared route/SEO metadata

4. Content Status Model

Status	Meaning	Production behavior
approved	RBB-approved public content	May render publicly if route/page requirements are satisfied
pending-review	Prepared but awaiting RBB review	Must not appear as approved public content
draft	Work in progress	Must not render publicly
archived	Previously approved but intentionally retired	Must not render publicly unless a specific archive experience is approved
placeholder	Structural empty-state content	May render only as a clearly identified placeholder/pending state

Important: a status field is an internal publishing control. It must not be presented to public visitors as editorial metadata unless specifically intended.

5. Approved-Only Publishing Rule

- Only records with status=approved may become public content records.
- Draft and pending records must not be rendered into public route output.
- Archived records must not remain accessible through active public slugs unless explicitly supported.
- Do not use CSS or client-side hiding as the publishing mechanism.
- Do not rely only on route-level noindex for draft protection.
- Do not put sensitive or confidential draft material into production content files simply because it is hidden from a page; build output and browser bundles can expose repository content.
- Prefer removing unapproved records from production content inputs entirely, or use a build-time filtering strategy that guarantees they are not emitted.

6. Content Model Principles

- Each content type has a stable, documented schema.
- Required fields are validated before build.
- Optional fields have explicit fallback behavior.
- Unknown fields should be treated deliberately rather than silently relied upon.
- References between content types use stable IDs/slugs rather than duplicated copy.
- Presentation components should not contain the source-of-truth content.
- Content modules should remain readable and maintainable by developers/editors working in the repository.

7. Stories Model

Continue the approved model established in Document 07:

id, slug, title, excerpt, category, date, author, featured, image, body, location, programId, projectId, impact, quotes, media, relatedStoryIds, status

- Categories remain News & Updates, Field Stories, Impact Stories and Community Stories unless RBB approves additional categories.
- Story slugs must be unique.

- Dates must use a consistent machine-readable format.
- Only approved stories can populate featured/latest/public detail experiences.
- References to programs/projects must resolve to valid approved records or be omitted.
- No invented author, location, quote or outcome.

8. Projects & Programs Model

Continue the project model established in Document 04:

{ id, title, slug, program, description, location, status, image, impact, partners }

- Program IDs must resolve to the four approved program areas.
- Project records require RBB-provided facts before publication.
- Do not populate location, partner, impact or status fields with assumptions.
- Projects without approved records should continue to use the existing empty state.
- Future project detail pages must be generated only for approved records.

9. Team Model

Continue the approved model from Document 10:

id, slug, name, role, status, category, department/area, short bio, biography, image, image alt, location, social links, related links, order, featured, approval date

- No invented people, titles, bios, locations, photos or social links.
- Only approved records can render public profiles.
- Team groups remain placeholders until RBB supplies the organization structure.
- Personal information should be limited to what RBB approves for publication.

10. Impact & Financial Content

- Impact metrics such as 50K+, 25 countries and 200+ partners remain pending RBB verification unless their status is explicitly approved.
- The conflicting 120+ figure remains excluded until clarified.
- 2026 financial percentages of 78% Programs & Services, 12% Fundraising and 10% Administration remain subject to the existing RBB verification state.
- Do not duplicate figures across multiple files when a shared source already exists.
- Financial and impact claims should have one canonical content source wherever practical.
- Changes to numerical claims require deliberate review because they can affect Home, Impact and Transparency simultaneously.

11. Contact, Donation & Policy Content

- Contact information remains absent until RBB provides verified contact details.
- Donation provider, currency, amounts, tax/receipt language and payment details remain pending until approved.
- Policy pages remain pending until RBB/legal review supplies approved text.
- Do not replace pending content with realistic-looking placeholders that could be mistaken for real operational information.

- Forms remain disabled until the prerequisites established in Documents 12 and 17 are satisfied.

12. Slugs & URL Stability

- Use lowercase, stable, human-readable slugs.
- Do not change a published slug casually.
- If an approved published slug must change, add an explicit permanent redirect from the old route.
- Avoid redirect chains.
- Do not create slugs containing private information.
- Slug uniqueness must be validated at build time within each content type and across routes where collisions are possible.

13. Content References

- Validate story → program/project references.
- Validate project → program references.
- Validate related-story references.
- Validate team/profile references used by About.
- Broken references should fail the build for approved content rather than silently producing broken public pages.
- Optional references may be omitted intentionally when the related content does not exist.

14. Images & Media

- Only approved RBB imagery may be published.
- Every meaningful image requires deliberate alt text.
- Decorative images must be explicitly marked decorative.
- Image paths must resolve at build time.
- Missing required images should fail validation or use a deliberate component-level fallback where the content model permits it.
- Do not add external media embeds without the technology/privacy review established in Document 17.
- Image optimization remains governed by Document 15.

15. SEO Content Requirements

- Public indexable content should have an appropriate title and description.
- Dynamic approved stories/projects/team profiles should derive metadata from the content record or approved SEO mapping.
- Do not duplicate identical descriptions across large numbers of pages.
- Canonical URLs must use the production-origin architecture established in Document 14/15.
- Draft/pending/unknown content must remain non-indexable and preferably not publicly routable.
- Do not add structured data to content types unless the underlying record is approved and the schema is appropriate.

16. Content Validation Layer

- Add build-time validation for required fields.

- Validate unique IDs and slugs.
- Validate enum values such as status/category/program.
- Validate internal references.
- Validate required images.
- Validate approved records have required public-facing fields.
- Fail the build for invalid approved records rather than shipping malformed public content.
- Keep validation messages understandable enough for the developer/editor maintaining the repository.

17. Build-Time Publishing

- Build-time rendering remains the publishing mechanism.
- Approved content enters the next production build.
- Content changes require a new build/deployment.
- Do not add a runtime database solely for content publishing.
- Do not add a CMS in this phase.
- Build reports should make it possible to see how many approved records were included by content type.
- Build output must not expose draft/pending content through public HTML, sitemap or structured data.

18. Content Governance Workflow

Recommended repository workflow:

- 1. RBB provides source material.
- 2. Developer/editor maps the material into the appropriate content model.
- 3. Record remains pending-review while being prepared.
- 4. Required fields, references, images and metadata are validated.
- 5. RBB approves the public content.
- 6. Status becomes approved.
- 7. Production build validates the record.
- 8. Pre-rendered output is generated.
- 9. QA checks the rendered page.
- 10. Deployment publishes the approved content.

19. Content Change Safety

- Treat public numerical claims, names, roles, partner names, locations and financial information as high-review fields.
- Keep source text traceable to RBB-provided material where practical.
- Do not silently rewrite facts to make them sound stronger.
- Do not convert uncertainty into certainty.
- When information conflicts, stop and request clarification rather than choosing a value.
- Keep old redirects when URLs change.

20. Future CMS Boundary

A CMS may be evaluated later if RBB needs non-developer publishing. That would be a separate architecture project.

- A future CMS would require authentication and role management.
- It would require an editorial workflow and permissions.
- It would require a database/API or managed content platform.
- It would require media management.
- It would require preview/publishing controls.
- It would require a migration strategy from repository content.
- It would require a new privacy/security assessment.
- None of those capabilities are part of Document 18.

21. Claude Code Implementation Prompt

You are implementing Document 18 – Content & Editorial Architecture for the Rising Beyond Borders website.

IMPORTANT SCOPE

This is FRONTEND / REPOSITORY-BASED content architecture only.

DO NOT create:

- a CMS
- an admin dashboard
- /admin routes
- login/authentication for editors
- a database
- a content API
- a browser content editor
- automatic publishing
- a new backend

CONTEXT

- Documents 01-17 are implemented and approved.
- The site uses src/content modules as its content source.
- Document 15 uses build-time pre-rendering.
- Document 17 requires no unapproved tracking.
- Do not invent RBB facts.
- Do not stage or commit changes.

GOAL

Formalize and validate the existing repository-backed content model so approved RBB content can be added safely and unapproved content cannot accidentally become public.

CONTENT STATUS

Support the established internal statuses:

- approved
- pending-review
- draft
- archived
- placeholder

RULE

Only approved records may become public content records.

Do not rely only on noindex or CSS hiding.

Do not ship sensitive/unapproved draft content in production bundles.

CONTENT MODULES

Preserve/reuse:

- homepage.js
- about.js
- work.js
- impact.js
- getInvolved.js
- stories.js
- contact.js
- transparency.js
- team.js
- donation.js
- policies.js
- seo.js

MODELS

Preserve the existing Story, Project and Team models from Documents 04, 07 and 10.

Do not invent fields that are not needed.

Keep references stable using IDs/slugs.

VALIDATION

Add build-time validation for:

- required fields
- unique IDs
- unique slugs
- status/category/program enums
- internal references
- required images
- approved-record requirements
- route/content collisions

FAIL THE BUILD when an approved record is malformed or references missing required content.

DRAFT/PENDING

Ensure draft/pending/archived records cannot appear in:

- public HTML
- sitemap

- structured data
- public story/project/team routes
- homepage featured/latest content

Do not expose sensitive draft text in browser bundles.

SLUGS

- lowercase
- stable
- human-readable
- unique
- permanent redirects for published slug changes
- no redirect chains
- no personal information in slugs

IMAGES

- approved imagery only
- deliberate alt text
- decorative flag where appropriate
- build-time path validation
- preserve Document 15 optimization
- do not introduce external media without Document 17 review

SEO

Approved public content must have appropriate metadata.

Draft/pending/unknown content remains non-indexable and preferably not routable.

Do not invent descriptions or structured-data facts.

SHARED SOURCES

Do not duplicate high-impact figures across unrelated files when a shared source already exists.

Existing RBB figures remain subject to their established verification states.

Do not reintroduce the excluded 120+ figure.

Do not invent contact, team, partner, project, financial or donation details.

BUILD REPORT

Extend the build/pre-render reporting only if useful to show:

- approved record counts by content type
- excluded draft/pending counts
- validation failures

Do not expose sensitive content in the report.

CONTENT WORKFLOW

Document the repository workflow:

source from RBB → pending-review → validation → RBB approval → approved → build → QA → deployment.

FUTURE CMS

Document that CMS/admin/database/editor capabilities are explicitly out of scope and may be evaluated as a separate future project.

VALIDATION

1. Production build passes with no warnings.
2. Existing routes remain intact.
3. Existing 48-page pre-render behavior remains intact.
4. Existing SEO/redirect behavior remains intact.
5. Draft/pending records do not appear publicly.
6. Invalid approved records fail the build.
7. References resolve correctly.
8. No sensitive/unapproved content is emitted.
9. Accessibility remains clean.
10. No analytics/tracking changes are introduced.
11. Dev server remains functional.
12. Do not stage or commit.

REPORT

Return:

- files created/modified
- content validation rules
- status behavior
- reference validation
- build-report changes
- tests run
- routes checked
- any issues found
- any RBB approvals still required.

22. Acceptance Checklist

- ✓ No CMS/admin/database/editor was created.
- ✓ Repository content remains the source of truth.
- ✓ Only approved records can become public.
- ✓ Draft/pending content cannot leak through HTML, sitemap or structured data.
- ✓ Required fields are validated at build time.
- ✓ IDs and slugs are unique and stable.
- ✓ Internal content references are validated.
- ✓ Images and alt text are validated appropriately.
- ✓ SEO metadata is handled deliberately.
- ✓ Existing RBB verification states are preserved.
- ✓ The 120+ figure remains excluded until clarified.
- ✓ Build-time publishing remains the mechanism.
- ✓ Content changes require rebuild/deployment.
- ✓ Future CMS work is explicitly separate.
- ✓ Existing SEO, performance, accessibility and observability behavior remains intact.

RBB decisions still required: editorial owner/approver, final content approval workflow, approved content records, approved imagery, verified impact/financial figures, team information, project information, contact information and any future decision to introduce a CMS.